

Assignment- Module-43

Collection Framework

Q.1 Ans- Collection Framework is a combination of classes and interface, which is used to store and manipulate the data in the form of objects. It provides various classes such as ArrayList, Vector, Stack, and HashSet, etc. and interfaces such as List, Queue, Set, etc. for this purpose.

Q.2 Ans- The ArrayList and LinkedList classes in Java are both part of the java.util package and implement the List interface, but they have significant differences in how they work, which makes them suitable for different use cases.

Feature	ArrayList	LinkedList
---------	-----------	------------

Data Structure	Dynamic array	Doubly linked list
----------------	---------------	--------------------

Access Time	$O(1)$ for random access	$O(n)$ for random access
-------------	--------------------------	--------------------------

Insertion/Deletion	$O(n)$ (except at end: $O(1)$)	$O(1)$ (after node is found)
--------------------	---------------------------------	------------------------------

Memory Usage	Less memory	More memory (due to pointers)
--------------	-------------	-------------------------------

Iteration Performance	Faster (better cache locality)	Slower (requires pointer traversal)
-----------------------	--------------------------------	-------------------------------------

Best Use Case	Frequent access by index	Frequent insertions and deletions at the beginning or middle
---------------	--------------------------	--

Q.3 Ans-

Feature	Iterator	ListIterator
Applicable Collections	Can be used to traverse any collection (e.g., Set , List , Queue , etc.).	Can only be used to traverse List collections (e.g., ArrayList , LinkedList).
Traversal Direction	Supports only forward traversal of elements.	Supports both forward and backward traversal of elements.
Modifying the Collection	Allows removal of elements during iteration using remove() method.	Allows modification of elements using set() , and supports add() and remove() .
Ability to Add Elements	Does not allow adding elements while iterating.	Allows adding elements while iterating using add() .
Access to Index	Does not provide access to the index of elements.	Provides access to the index of elements using nextIndex() and previousIndex() .
Methods for Traversal	- hasNext() hasNext()) - next() - remove() - hasPrevious() () - previous() - remove() - set() - add()	
Direction-Specific Methods	Does not have direction-specific methods.	Has direction-specific methods like hasPrevious() and previous() .

Use Case	Suitable for collections where only forward traversal and element removal are required.	Suitable for List collections where bi-directional traversal, addition, removal, and modification are needed.
----------	---	--

-

Q.4 Ans Feature	Iterator	Enumeration
Applicability	Can be used with all collections in the Java Collection Framework (e.g., List , Set , etc.).	Primarily used with legacy classes like Vector and Hashtable .
Traversal Direction	Supports only forward traversal of elements.	Supports only forward traversal of elements.
Element Removal Support	Allows element removal during iteration using the remove() method.	Does not support element removal.
Method Names	- hasNext() - next() - remove()	- hasMoreElements() - nextElement()
Type-Safety	Introduced with Java Collections Framework, making it type-safe with generics.	Does not support generics, hence not type-safe.
Usage in Modern Applications	Widely used and preferred for modern Java applications.	Considered outdated and rarely used in modern Java applications.
Performance	Generally slightly slower due to additional features like element removal.	Slightly faster but lacks features like removal.
Introduced In	Java 2 (part of the Collections Framework).	Java 1.0 (part of legacy collection classes).
Use Case	For iterating over any modern Java collection.	For iterating over legacy classes like Vector and Hashtable .

Q.5 Ans: The List and Set both extend the collection interface. However, there are some differences between the two

which are listed below@

- o The List can contain duplicate elements whereas Set includes unique items@

- o The List is an ordered collection which maintains the insertion order whereas Set is an unordered collection

which does not preserve the insertion order@

- o The List interface contains a single legacy class which is Vector class whereas the Set interface does not

have any legacy class@

- o The List interface can allow a number of null values whereas Set interface only allows a single null value.

Q.6 Ans- Ans: Both HashSet and TreeSet are implementations of the Set interface in Java, but they have some

differences in terms of their properties and usage1

- o Ordering: HashSet is an unordered collection of elements, while TreeSet is a sorted set of elements based on their natural order or a custom comparator

- o Duplication: HashSet does not allow duplicate elements, while TreeSet does not allow duplicates as well

- o Implementation: HashSet is implemented using a hash table, while TreeSet is implemented using a self-balancing binary search tree (Red-Black tree)

- o Performance: HashSet has constant-time complexity $O(1)$ for adding, removing, and testing the existence of

an element, while TreeSet has a logarithmic-time complexity $O(\log n)$ for these operations due to the self-balancing property

- o Memory usage: HashSet uses less memory than TreeSet because it only stores the elements, while TreeSet stores additional information for maintaining the order

- o Iteration: HashSet provides no guarantees regarding the order of iteration, while TreeSet guarantees the elements are iterated in sorted order

- o Usage: HashSet is suitable when ordering is not important, and fast access and membership tests are needed. TreeSet is suitable when elements need to be sorted or accessed in a order.

Q.7Ans: Both arrays and ArrayLists are used to store collections of elements in Java, but they have some differences in terms of their properties and usage

- o Type: Arrays can store elements of primitive data types as well as objects, while ArrayList can only store objects

- o Size: The size of an array is fixed once it is created, while the size of an ArrayList can be dynamically increased or decreased by adding or removing elements

- o Mutability: Arrays are mutable, meaning that you can modify the elements in an array after it has been created. ArrayList is also mutable, but the only way to modify it is by adding, removing or modifying elements

- o Performance: Arrays have better performance than ArrayLists for certain operations, such as accessing elements by index, because they are implemented as a continuous block of memory. ArrayLists, on the other hand, use dynamic memory allocation and are implemented as a dynamic array, which may result in more memory overhead and slower performance for certain operations

- o Methods: Arrays have a limited set of methods compared to ArrayLists, which provides more methods for manipulating the collection, such as adding, removing, and sorting elements.